TESI DI LAUREA MAGISTRALE IN INFORMATICA

Il problema del Neighborhood Search su grafi temporali

Franco Salvucci

Relatore: Prof. Luciano Gualà Correlatore: Dott. Alessandro Straziota

Università degli Studi di Roma "Tor Vergata"

A.A. 2025/2026

Percorso della presentazione

01
Fondamenti
Definizioni e preliminari

02
Soluzioni proposte
Le due varianti su grafi temporali

03
Esperimenti
Risultati e confronti

04
Conclusioni
Contributi e sviluppi

SEZIONE 01

Fondamenti e definizione dei problemi

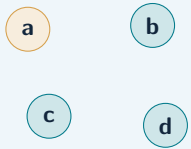
Neighborhood Search statico, MinHash/LSH e passaggio alla dimensione temporale

Neighborhood Search sui grafi statici

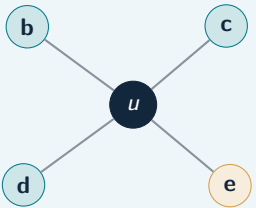
Formulazione del problema: Dati $G = (V, E)$, $S \subseteq V$, una soglia $\tau \in [0, 1]$ e una funzione di similarità $\sigma : 2^V \times 2^V \to [0, 1]$ trovare un nodo $u \in V \setminus S$ tale che

$$\sigma(S, N(u)) \geq \tau.$$

Insieme della query S



Vicinato del candidato u



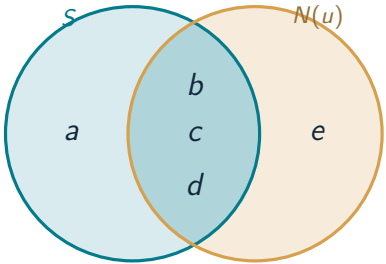
$S = \{a, b, c, d\}$

$N(u) = \{b, c, d, e\}$

La similarità scelta: indice di Jaccard

Dalla formulazione generica $\sigma(S, N(u))$ alla misura insiemistica

$$\sigma(S, N(u)) = J(S, N(u)) = \frac{|S \cap N(u)|}{|S \cup N(u)|}$$



Esempio

$$J(S, N(u)) = \frac{3}{5} = 0.6$$

Tre elementi in comune su cinque distinti.

MinHash

$\Pr[h_{\min}(S) = h_{\min}(N(u))] = J(S, N(u))$

k funzioni $\rightarrow$ signature compatta.

Neighborhood Search mediante MinHash e LSH

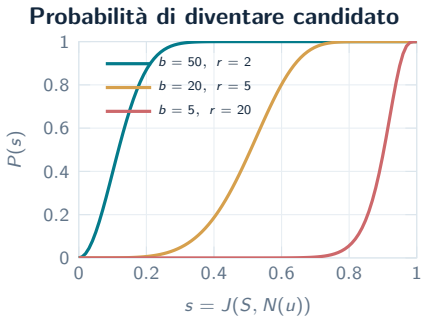
1. Sketch MinHash di S e di ogni $N(u)$

2. Suddivisione in b bande di r righe

3. Bucket LSH: almeno una banda uguale

4. Verifica finale sui soli candidati

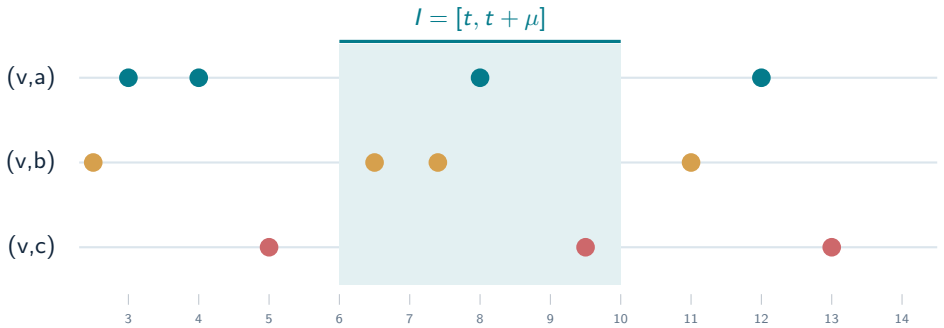
La verifica può calcolare Jaccard esatta o stimarla dalla firma completa.



$k = br$; $P(s) = 1 - (1 - s^r)^b$.

Un'ulteriore formulazione del problema prevede l'utilizzo di un singolo nodo v al posto di S. Per semplicità di trattazione, da qui in avanti useremo $S = v$

Grafi temporali: il vicinato dipende dall'intervallo



$N^I(v) = \bigcup_{t \in I} N^t(v)$

$\Lambda(v) = \{t : \ v \textbf{ è attivo in } t\}$

Un arco temporale è rappresentato dalla tripla (u, v, t); il vicinato può cambiare da una finestra all'altra.

Due varianti dello stesso problema

Dato $\mathcal{G} = (V, E, \lambda), v \in V, \tau \in [0, 1]$ **e** $\sigma : 2^V \times 2^V \to [0, 1]$ **trovare** $u \neq v$ **ed un intervallo** $I = [t, t + \mu]$ **tali che** $J(N^I(v), N^I(u)) \geq \tau$

VARIANTE 1

Ampiezza fissa

μ viene passato come **input del problema**

Indice progettato per quella finestra

VARIANTE 2

Ampiezza variabile

μ fornito con **la query**

Serve supportare più ampiezze

SEZIONE 02

Soluzioni proposte

Pruning, indicizzazione LSH modificata e oracolo $\mathcal{O}$

Variante 1: risultati teorici della soluzione

μ è fissata in preprocessing; $w_v = |W(v)|$, $m_v = |\Lambda(v)|$,
$C_v = \sum_{t \in W(v)} |C(v,t)|$ è il totale dei candidati esaminati.

Building time

$$O\left(\sum_{v \in V} w_v(k \log m_v + b)\right)$$

Costruzione indice LSH

Spazio

$$O\left(b \sum_{v \in V} w_v\right)$$

Memoria indice LSH

Query time

$$O(w_v(k \log m_v + b) + kC_v)$$

Per una query su v

$$w_v \leq \min\{T - \mu, (\mu + 1)|\Lambda(v)|\}$$

Bound dell'indice LSH con pruning; la costruzione e lo spazio dell'oracolo $\mathcal{O}$ sono contabilizzati separatamente.

Variante 1: due ingredienti fondamentali

01 INDICE LSH UNIFICATO

Pruning temporale $W(v)$

Un unico insieme di tabelle con chiave

$\big(\mathrm{Hash}_j(B_j(v,t)), t\big)$

Riduce finestre e confronti

02 ORACOLO $\mathcal{O}$

Recupero degli sketch temporali

Range Tree sugli istanti attivi

$\mathrm{Sketch}(N^{[t,t+\mu]}(v))$

Combina sketch precomputati

Query su v → Sketch tramite $\mathcal{O}$ → Candidati via LSH → Verifica

Variante 1: il pruning temporale $W(v)$

$W(v) = \{t : [t, t+\mu] \cap \Lambda(v) \neq \varnothing\}$ Esempio $\mu = 2$

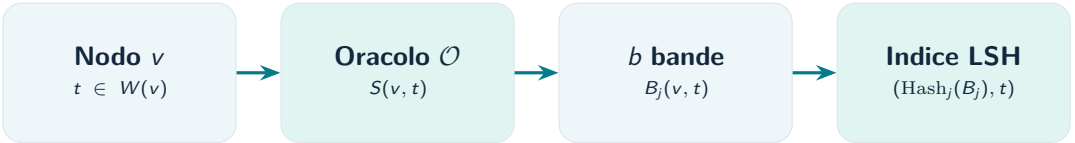


Finestre senza eventi **escluse**

Finestre con eventi **mantenute**

Ogni istante attivo genera al più $\mu + 1$ partenze rilevanti: $|W(v)| \leq \min\{T - \mu, (\mu + 1)|\Lambda(v)|\}$.

Variante 1: l'indice LSH unificato



Indice globale

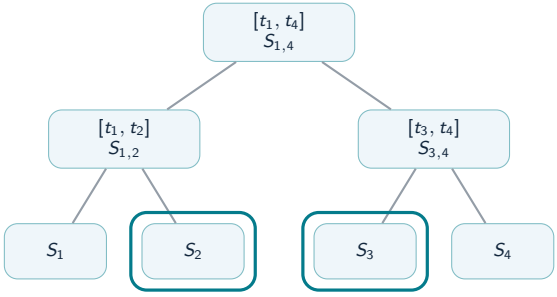
Tutte le coppie (v, t) rilevanti condividono le stesse b tabelle.

Timestamp nella chiave

Si confrontano candidati della **stessa finestra**.

Filtro LSH $\longrightarrow$ confronto delle signature complete

Oracolo $\mathcal{O}$: recupero efficiente degli sketch



Query $[t_2, t_3]$: merge di S_2 e S_3

Range Tree

sulle date attive $\Lambda(v)$

Sketch MinHash

componibili via minimi

$O(k \log |\Lambda(v)|)$

Recupero dello sketch

Oracolo $\mathcal{O}$: building $O(k \sum_v m_v \log m_v)$; spazio $O(s \sum_v m_v)$ per sketch di dimensione s.
Con 1-MinHash, una RMQ recupera lo sketch in $O(1)$ (spazio $O(T)$ per nodo).

Variante 2: ampiezza nota soltanto alla query

Difficoltà

$\Theta(T^2)$ intervalli complessivi

Un indice LSH per ogni μ sarebbe costoso.

Risultato proposto

Filtro euristico sulla **prima componente MinHash**.

Corrispondenze $\mathbf{MinHash}_1$ sulle finestre

↓

Rettangoli geometrici associati ai nodi

↓

Range Tree multidimensionali $\rightarrow$ candidati

↓

Verifica successiva con firma completa

Variante 2: bound dell'oracolo sul primo MinHash

$N \leq nT$: rettangoli indicizzati; $k_v(\mu)$: rettangoli rilevanti per la query;
$M = |C(v, \mu)|$: coppie nodo–finestra riportate dal filtro.

Building time

$$O\big(|\mathcal{E}| + nT + N \log^3 N\big)$$

Costruzione rettangoli + Range Tree 4D

Spazio

$$O\big(nT + N \log^3 N\big)$$

Indice geometrico

Query del filtro

$$O\big(k_v(\mu) \log^3 N + M\big)$$

Prima componente MinHash

$$O\big(T \log^3(nT) + M\big)$$

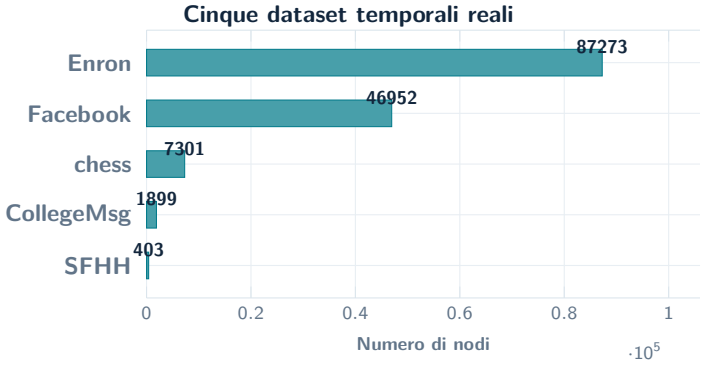
Bound semplificato della ricerca dei candidati. Non comprende la verifica finale con le signature complete, né lo spazio dell'oracolo $\mathcal{O}$ usato per recuperarle.

SEZIONE 03

Valutazione sperimentale

L'oracolo, il pruning e i confronti con la Brute Force

Configurazione degli esperimenti



Parametri principali

$\tau = 0.35$
$k = 256$
$b = 8,\ r = 32$

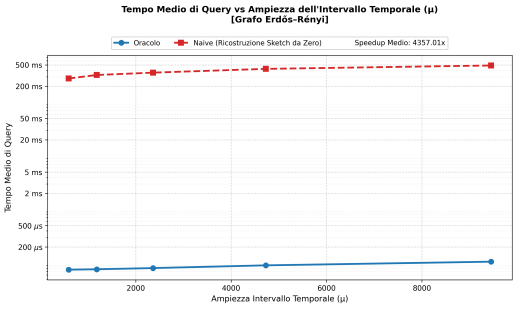
Strategie a confronto

BF
LSH+W
LSH–W
Alt+W

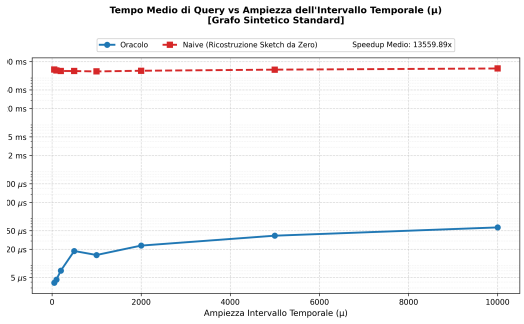
Fino a 8000 sorgenti distinte; le quattro strategie condividono lóracolo $\mathcal{O}$.

L'oracolo $\mathcal{O}$ evita la ricostruzione del vicinato

Erdős–Rényi (10 000 nodi)



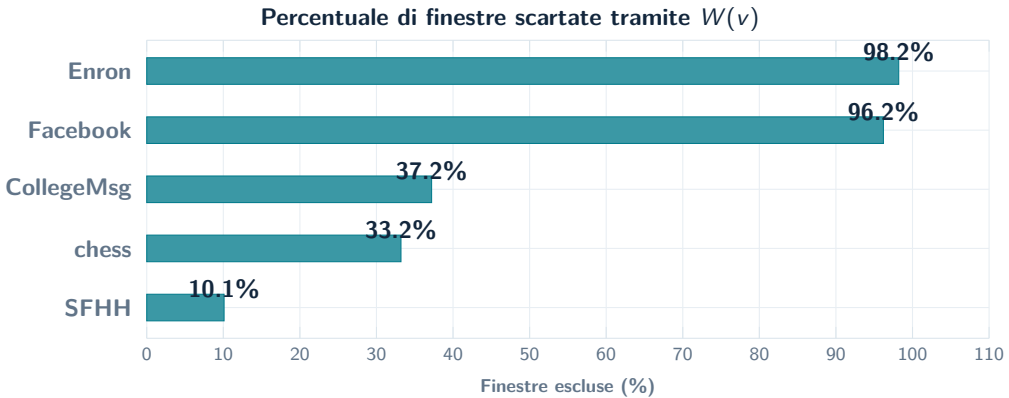
Grafo sintetico (300 000 nodi)



Sketch tramite oracolo: microsecondi. Ricostruzione naïve: millisecondi.

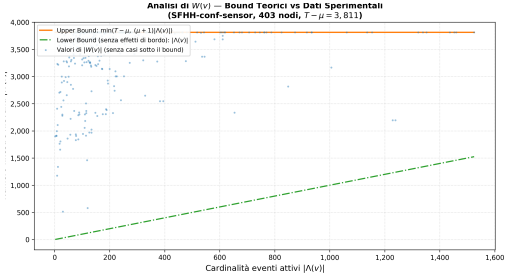
Figure estratte dalla tesi, Fig. 7; il confronto riguarda il recupero degli sketch.

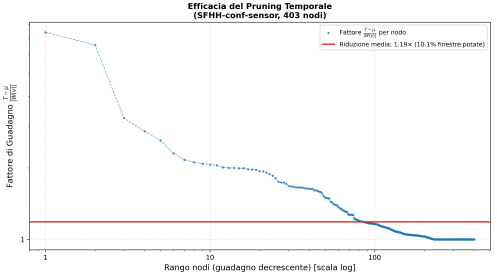
Pruning: quante finestre vengono eliminate?



Fonte: Tabella 2 e Fig. 8 della tesi. Il pruning non garantisce lo stesso guadagno su ogni distribuzione temporale.

Pruning moderato: SFHH-conf-sensor

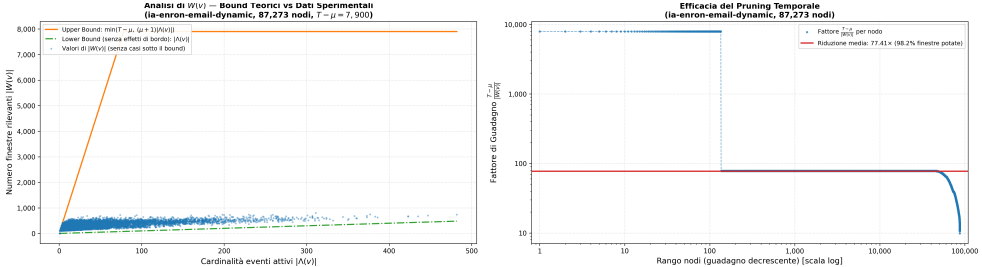




Solo il 10.1% delle finestre eliminate **Guadagno medio** $1.19\times$

Figura 8(b) della tesi. Con eventi distribuiti densamente, $W(v)$ rimane vicino all'insieme delle finestre ammissibili.

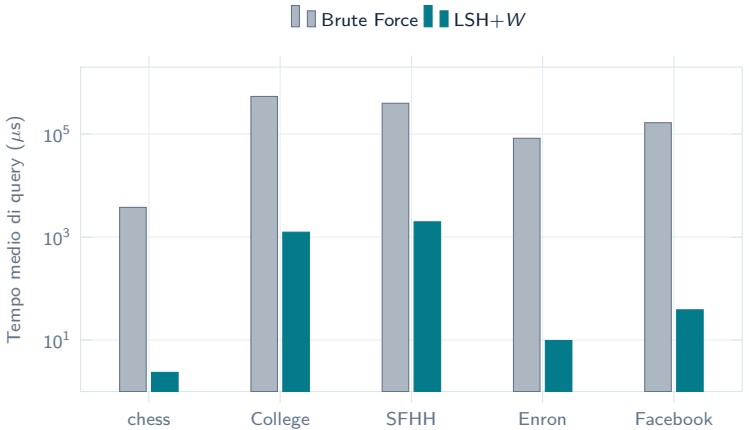
Pruning molto efficace: ia-enron-email-dynamic



98.2% delle finestre eliminate **Guadagno medio** $77.41\times$

Figura 8(d) della tesi. Con attività temporalmente sparsa, la ricerca scarta la grande maggioranza delle finestre.

Tempi di query: LSH+W contro Brute Force



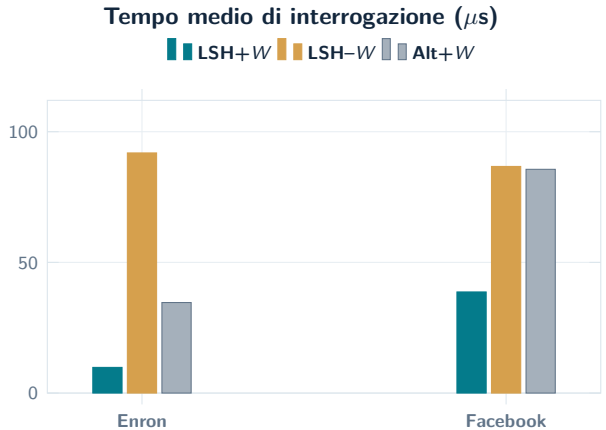
8431×
speedup Enron

4287×
speedup Facebook

1596×
speedup chess

Tabella 3. Query booleana, arresto al primo risultato positivo; asse dei tempi in scala logaritmica.

Confronto fra le tre strategie LSH



LSH+W

Indice globale e pruning.

LSH−W

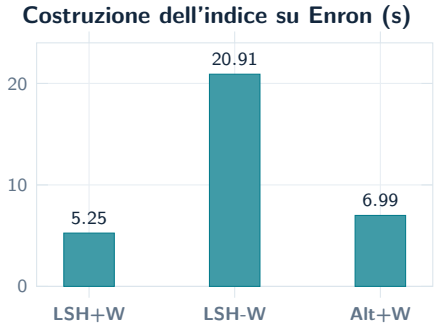
Stesso indice senza pruning.

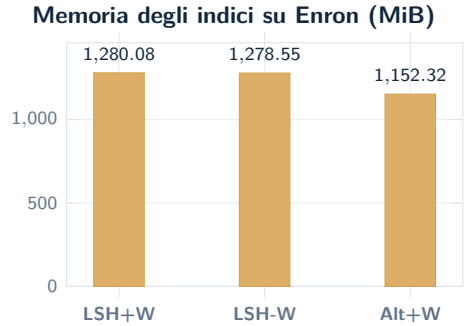
Alt+W

Indici separati per finestra.

Tabella 3. In SFHH il pruning non riduce il tempo di query: il vantaggio dipende dai dati.

Il costo del preprocessing e della memoria

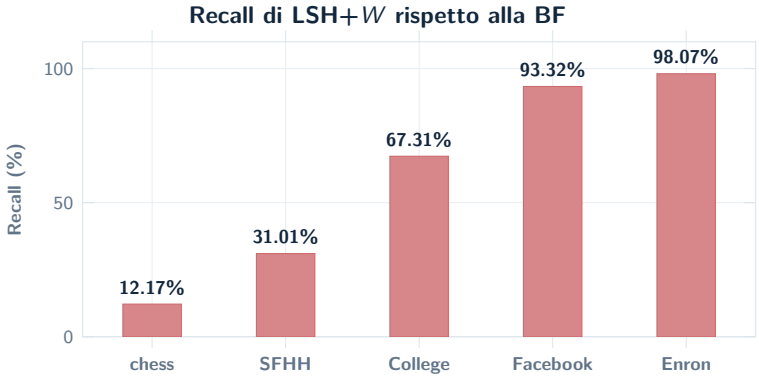




L'indicizzazione accelera la query, ma richiede costruzione e RAM.

Tabella 4. I valori non includono la memoria e la costruzione dell'oracolo $\mathcal{O}$, comuni alle strategie.

La velocità introduce un compromesso: la recall



Confronto relativo alle stesse signature MinHash (non al Jaccard esatto). Tab. 5 della tesi.

0

falsi positivi vs BF

Falsi negativi

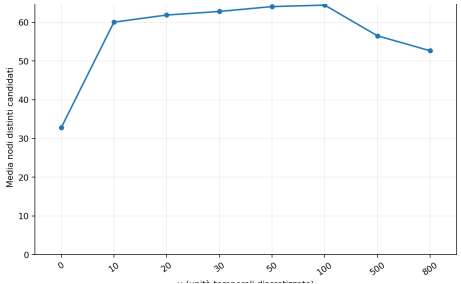
Alcuni nodi simili non diventano candidati LSH.

Recall

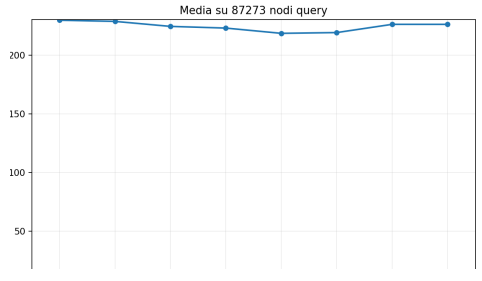
$$\text{Recall}_{\text{LSH}|\text{BF}} = \frac{|B \cap L|}{|B|}, \quad |B| > 0.$$

Variante 2: il filtro geometrico riduce i candidati

CollegeMsg



Enron: 87 273 **nodi**



Su Enron circa 220–230 candidati distinti in media.

Fig. 11: è misurato soltanto il filtro preliminare $|C_1(v, \mu)|$, non la verifica completa, la recall o la velocità dell'intera query.

SEZIONE 04

Conclusioni e sviluppi futuri

Contributi, limiti sperimentali e direzioni di ricerca

Risultati principali e prossimi sviluppi

Pruning temporale

98.2%

finestre escluse su Enron

Velocità della query

8431×

speedup massimo rilevato

Compromesso recall

12–98%

recall dipendente dal dataset

Sviluppi futuri

Esplorare altri framework *Filter&Verify* • Valutare la verifica completa nella variante 2 • Supportare aggiornamenti dinamici



Grazie per l'attenzione!

Franco Salvucci

Il problema del Neighborhood Search su grafi temporali